

Emotion Detection & Learning Support Engine

Project Description:

Emotion Detection & Learning Support Engine is an AI-powered web application designed to identify students' learning emotions from free-text learning problems and provide personalized educational guidance. The platform compares predictions from a fine-tuned BERT model and a Multi-label BiLSTM network, detects both primary and mixed emotions, and generates study guidance tailored to how the student is feeling.

Built using Python, Streamlit, TensorFlow/Keras, Hugging Face Transformers, and Google Gemini AI, the application enables students to receive emotion-aware support while studying. The system also maintains session history, tracks emotion analytics, and generates downloadable AI reports for future reference.

Scenarios

Scenario 1:

A college student studying a difficult Data Structures topic enters their learning problem into the application. The system detects that the student is confused and frustrated, compares BERT and BiLSTM predictions, and generates a personalized explanation with study tips using Google Gemini AI.

Scenario 2:

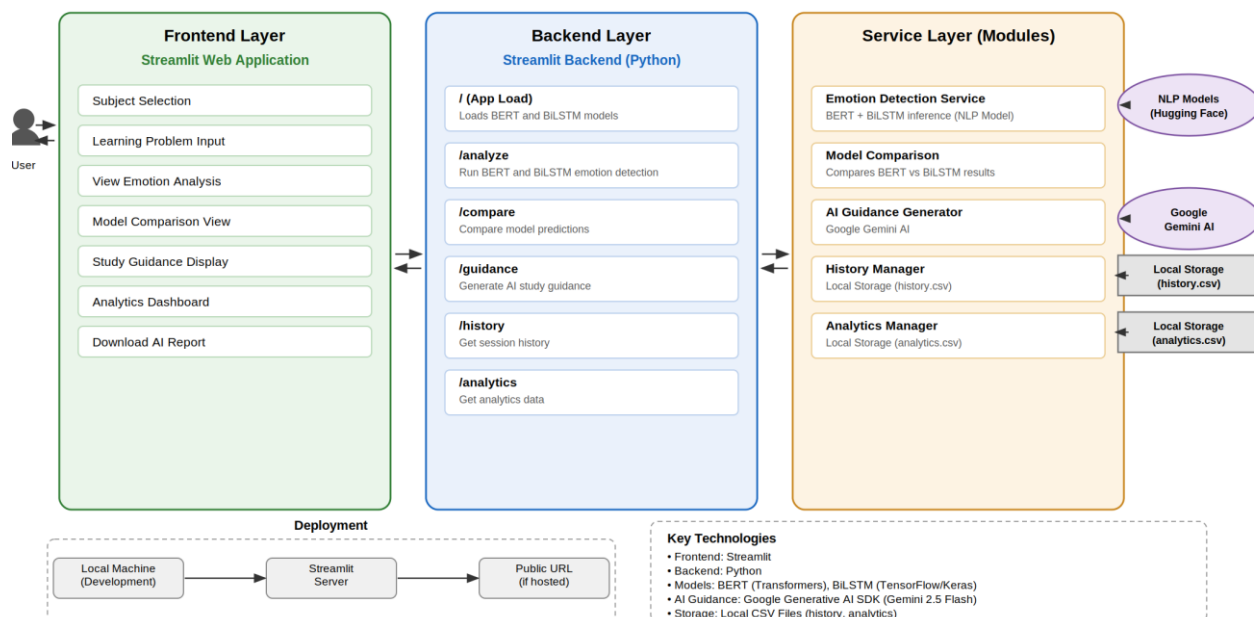
A school student preparing for a Mathematics exam describes feeling anxious about a topic. The system identifies the primary emotion as anxious with confidence scores, and generates a motivating study plan to help the student build confidence before the exam.

Scenario 3:

A self-learner working through an online Physics course enters a learning problem and receives AI-generated guidance, a comparison between BERT and BiLSTM emotion predictions, and a downloadable study report for future reference.

Technical Architecture

Technical Architecture: Emotion Detection & Learning Support Engine



Description:

The Emotion Detection & Learning Support Engine follows a modular three-tier architecture.

- The Frontend Layer consists of a Streamlit-based user interface.
- The Backend Layer is implemented in Python and handles model loading, inference requests, and routing.
- The Service Layer contains reusable modules for emotion detection, model comparison, AI guidance generation, history logging, and analytics.

The architecture enables scalability, maintainability, and modular development.

Pre-requisites

- Python Programming Knowledge
- Streamlit Framework Knowledge
- Deep Learning Concepts (BERT, BiLSTM)
- Natural Language Processing Concepts
- Hugging Face Transformers Library
- TensorFlow / Keras Basics
- Google Generative AI SDK / REST API Concepts
- Git and GitHub Basics
- Virtual Environment Management

Project Workflow**Milestone 1: Environment Setup and Architecture Design****Activity 1.1: Define Project Requirements**

- Identify student and learner requirements.
- Define learning-emotion use cases.
- Finalize project scope.

Activity 1.2: Design System Architecture

- Create architectural diagrams.
- Define interactions between frontend, models, and services.

Activity 1.3: Set Up Development Environment

- Install Python.
- Create virtual environment.
- Install dependencies.

Activity 1.4: Create Project Structure

- Create folders for:
 - app/
 - services/
 - models/
 - utils/
 - frontend/
 - tests/
-

Milestone 2: Model & Backend Development**Activity 2.1: Implement Streamlit Backend**

Create core application functions:

```
load_models()
analyze_input()
```

Activity 2.2: Develop Emotion Detection Module (BERT)

Implement BERT-based emotion prediction using Hugging Face Transformers.

Functions:

```
predict_emotion_bert()
```

Responsibilities:

- Predict the primary and secondary emotion.
- Return confidence scores and emotion probabilities.

Activity 2.3: Develop Emotion Detection Module (BiLSTM)

Functions:

predict_emotion_bilstm()

Responsibilities:

- Detect mixed emotions.
- Return confidence scores for each emotion.

Activity 2.4: Develop AI Guidance Module

Integrate the Google Generative AI SDK (Gemini 2.5 Flash).

Functions:

generate_study_guidance()

Responsibilities:

- Generate topic explanations and study tips.
- Generate a personalized study plan and motivation message.

Milestone 3: Frontend Development

Activity 3.1: Develop Streamlit Interface

Create user-friendly pages for:

- Subject selection.
- Learning problem input.
- Emotion analysis display.
- Model comparison display.
- Study guidance display.

Activity 3.2: Add User Interaction Features

Implement:

- Session history view.
- Emotion analytics dashboard.
- AI report download.
- Loading animations.

Milestone 4: Data Storage and Logging

Activity 4.1: Session History

Store generated sessions in:

history.csv

Responsibilities:

- Save detected emotions and generated guidance.
- Retrieve historical sessions.

Activity 4.2: Analytics Logging

Store analytics data in:

analytics.csv

Responsibilities:

- Track emotion trends across sessions.
- Track subject-wise statistics for the dashboard.

Milestone 5: Testing and Validation

Activity 5.1: Unit Testing

Create test files:

```
test_emotion_detection.py  
test_guidance_generator.py  
test_analytics.py
```

Use:

```
pytest -v
```

Validate:

- Model loading and inference.
- Emotion prediction accuracy on sample inputs.
- AI guidance generation.
- Analytics and history storage.

Activity 5.2: Application Testing

Access the application locally:

<http://localhost:8501>

Test:

- Subject selection and problem input flow.
- Emotion detection and model comparison.
- Study guidance generation.
- Analytics dashboard.

Milestone 6: Deployment and Documentation

Activity 6.1: Local Deployment

Run Streamlit:

```
streamlit run app.py
```

Activity 6.2: Version Control

Upload project to GitHub.

Repository includes:

- Source code.
- Documentation.
- Images.
- Tests.
- README.

Exploring the Application Pages

Home Page

Description:

The homepage allows users to:

- Select an academic subject.
- Enter their learning problem.
- Generate an emotion analysis.

Emotion Analysis Section

Description:

Displays the primary and mixed emotions detected from the student's learning problem, along with confidence scores from both BERT and BiLSTM.

Example:

Confused (72%)

Frustrated (18%)

Curious (10%)

Study Guidance Section

Description:

Displays the AI-generated study guidance based on the detected emotions.

Example:

"It looks like you're finding recursion confusing. Let's break it down with a simpler example and a step-by-step explanation..."

Analytics Dashboard

Description:

Displays emotion trends across sessions and subjects, along with the most frequent emotion and average confidence score.

History Section

Description:

Displays previously analyzed learning sessions along with the guidance generated for each.

Conclusion

Emotion Detection & Learning Support Engine successfully demonstrates how Artificial Intelligence and Natural Language Processing can improve the learning experience. The system assists students by detecting their learning emotions, comparing BERT and BiLSTM model predictions, generating personalized study guidance, and maintaining session history and analytics.

The modular architecture built using Streamlit, TensorFlow/Keras, and Hugging Face Transformers ensures maintainability and scalability. Future enhancements include multilingual emotion detection, voice and facial emotion recognition, LMS integration, and a teacher dashboard for tracking student progress.

This project highlights the practical application of AI in enhancing personalized education, student engagement, and emotional well-being while learning.